

План урока C++: восстановление после перерыва

Ученица: Дарья. Последний урок: 06.06.2026. Текущий урок: 23.06.2026. Перерыв: 17 дней.

Цель занятия

Не пытаться сразу идти дальше. Цель - за один урок вернуть рабочую базу: чтение кода, ввод/вывод, условия, switch и циклы. В конце занятия ученица должна сама написать 3 короткие программы: обратный отсчет, сумма чисел, меню-светофор в цикле.

Что видно по Git

Факт	Что это значит для урока
05.06 - создана домашка	Темы домашки: ввод/вывод, условия, switch, for, while, break, continue, накопление суммы.
06.06 - коммит ученицы	Ученица попробовала почти все задания, но в циклах оставила несколько пометок "Есть вопрос". Это главный сигнал: забыты не команды, а модель работы цикла.
Текущие локальные правки	Исправлены счетчики, условия циклов, ввод внутри цикла, break при делении на ноль, выход из while после switch. Файл подключен как loops() через .h и CMake.

Диагностика: что она забыла

- Цикл всегда имеет 3 части: стартовое значение, условие продолжения, изменение счетчика.
- Счетчик должен двигаться в сторону завершения: при обратном отсчете нужно --N, а не ++N.
- Если нужно несколько попыток, ввод должен быть внутри цикла, иначе программа проверяет одно и то же значение бесконечно.
- Для суммы нужен накопитель: `sum = 0` до цикла, `sum += i` внутри цикла, `cout` после цикла.
- Оператор `=` присваивает, а `<=` сравнивает. Ошибка `number = N` в условии цикла ломает логику.
- `break` внутри `switch` выходит только из `switch`. Чтобы выйти из `while`, нужна отдельная проверка после `switch` или другой флаг.
- `continue` пропускает остаток текущего круга цикла. Его нельзя ставить просто "для красоты".

Как начать урок

Сказать спокойно: "После 17 дней нормально забыть синтаксис. Сегодня не проверка памяти, а сборка заново. Нам важно не заучить команды, а понять схему: данные вошли, условие проверилось, цикл повторился".

Сценарий урока на 80-90 минут

Время	Что делать	Результат
0-10 мин	Быстрый запуск: открыть проект, найти homework_2026_06_06.cpp, показать main(), cout, cin, переменные.	Она снова видит структуру программы.
10-20 мин	Повторить if / else и switch на светофоре без цикла.	Понимает выбор одного варианта.
20-40 мин	Разобрать циклы через "3 кита": старт, условие, шаг. Сначала for, потом while.	Понимает, почему цикл начинается и заканчивается.
40-60 мин	Разобрать ее ошибки из домашки: задания 2, 3, 4, 5, 8.	Ошибки становятся учебным материалом.
60-75 мин	Дать 3 мини-задачи на занятии: обратный отсчет, сумма, угадай число.	Проверка, что она может писать сама.
75-90 мин	Финальный блок: switch внутри while. Светофор до q.	Понимает разницу break из switch и break из while.

Блок 1. Скелет программы

Попросить ее вслух объяснить каждую строку. Если путается - не исправлять сразу, а задать вопрос: "Где программа начинается? Где заканчивается? Где ввод? Где вывод?"

```
#include <iostream>

int main()
{
    int n;
    std::cin >> n;
    std::cout << n << std::endl;
    return 0;
}
```

Блок 2. Условия

Главная мысль: if отвечает на вопрос "правда или нет", switch выбирает один вариант по конкретному значению.

```
if (n % 2 == 0) {
    std::cout << "Четное" << std::endl;
} else {
    std::cout << "Нечетное" << std::endl;
}

switch (symbol) {
    case 'r':
        std::cout << "Стоять" << std::endl;
        break;
    default:
        std::cout << "Неизвестный сигнал" << std::endl;
        break;
}
```

Блок 3. Циклы - главная тема урока

Объяснить через одну фразу: цикл - это повторение, которое должно знать откуда начать, когда остановиться и как изменяться.

Часть цикла	Вопрос ученице	Пример
Старт	С какого числа начинаем?	int i = 1
Условие	Пока что повторяем?	i <= N
Шаг	Как приближаемся к концу?	++i или --i

Разбор ее ошибок и исправлений

1. Обратный отсчет: счетчик должен уменьшаться, иначе цикл не закончится.

```
int N;
std::cin >> N;

while (N >= 1) {
    std::cout << N << std::endl;
    --N;
}
std::cout << "Старт" << std::endl;
```

2. Сумма: печатать нужно не каждое число, а накопленный результат после цикла.

```
int N, sum = 0;
std::cin >> N;

for (int i = 1; i <= N; ++i) {
    sum += i;
}
std::cout << sum << std::endl;
```

3. Четные числа: сначала решаем, что пропустить, потом печатаем.

```
for (int number = 1; number <= N; ++number) {
    if (number % 2 != 0) {
        continue;
    }
    std::cout << number << std::endl;
}
```

4. Угадай число: ввод должен повторяться внутри while.

```
int secret = 7;
int guess;

while (true) {
    std::cin >> guess;

    if (guess < secret) {
        std::cout << "Больше" << std::endl;
    } else if (guess > secret) {
        std::cout << "Меньше" << std::endl;
    } else {
        std::cout << "Угадала" << std::endl;
        break;
    }
}
```

Switch внутри while: важный момент

Это самый важный пример на сегодня. Здесь легко ошибиться: break в case q выйдет из switch, но не из while. Поэтому после switch нужна отдельная проверка symbol == q.

```
char symbol;

while (true) {
    std::cin >> symbol;

    switch (symbol) {
        case 'r':
            std::cout << "Стоять" << std::endl;
            break;
        case 'y':
            std::cout << "Приготовиться" << std::endl;
            break;
        case 'g':
            std::cout << "Ехать" << std::endl;
            break;
        case 'q':
            std::cout << "Выход" << std::endl;
            break;
        default:
            std::cout << "Неизвестный сигнал" << std::endl;
            break;
    }

    if (symbol == 'q') {
        break;
    }
}
```

Контрольные вопросы по ходу урока

- Что будет, если в обратном отсчете написать ++N?
- Почему sum нужно создавать до цикла?
- Почему cout суммы стоит после цикла?
- Чем = отличается от == и <=?
- Из чего выходит break, если он стоит внутри switch?
- Где должен быть cin, если пользователь вводит много попыток?

Мини-практика на занятии

Задача	Условие	Что проверяет
1	Ввести N и вывести от N до 1.	while, --N, условие остановки
2	Ввести N и вывести сумму от 1 до N.	for, накопитель sum
3	Вводить символы светофора, пока не введено q.	while true, switch, break

Что не делать на этом уроке

- Не давать новые темы: массивы, функции, строки, классы.
- Не требовать идеальный синтаксис с первой попытки. Сначала добиваться правильной логики.
- Не решать за нее сразу. Лучше задавать вопросы: "Что изменится после этого круга?", "Когда цикл остановится?"
- Не пропускать ее пометки "Есть вопрос". Это лучшие точки для повторения.

Домашка после урока

Дать короткую домашку, не больше 4 задач: обратный отсчет, сумма, угадай число, светофор до q.
Требование: каждую задачу запустить и написать в комментарии 2 тестовых ввода.

Итоговая фраза для ученицы

"Сегодня мы восстановили не отдельные команды, а схему мышления: программа получает данные, проверяет условия и повторяет действия через цикл. Если ты понимаешь старт, условие и шаг, ты можешь написать любой простой цикл".